Sky Chess Lab

Designing an embodied AI chess partner for a physical board

Research status	Interaction model and chess engine: functional prototype. Monocular camera move recognition: unresolved research problem. **Repository:** github.com/fa366193/sky-chess-lab

Abstract

Sky Chess Lab asks whether a laptop can become the social half of an over-the-board chess game. The system pairs a physical chessboard with Sky, an animated conversational character who teaches, reacts, chooses moves, and maintains the game state. This paper documents the design hypothesis, implementation, experiments, failure modes, and next technical architecture. The central finding is negative but useful: aggregate per-square appearance differences are not sufficient for dependable move recognition under uncontrolled room lighting and a moving hand. A robust version should combine geometric calibration, explicit occupancy classification, temporal hand-state segmentation, confidence-aware legal-move inference, and a human recovery path.

The research question

> *Can an AI opponent feel physically co-present without replacing the analog chessboard with a digital board?*

Design constraint	The screen never displays a playable chessboard. The physical board remains the shared object; the interface contains Sky, her desk, dialogue, move instructions, and system status.

1. Interaction thesis

Most chess software asks the player to enter a digital environment. Sky Chess Lab reverses that relationship: computation watches the player's existing physical practice. Sky is designed as a situated partner rather than a dashboard. Her idle motion, speech, turn-taking, and reactions are not decoration; they are the interface through which the system explains uncertainty and coordinates physical actions.

Experience loop

01	Frame	User positions the laptop so all 64 squares are visible.
02	Calibrate	User identifies the four inner board corners and verifies orientation.
03	Choose	User selects White or Black; Sky takes the other side.
04	Play	Camera observes a settled physical move; chess rules constrain interpretation.
05	Respond	Stockfish selects Sky's move; Sky explains it and waits for physical synchronization.

Embodiment cues

- Continuous breathing, micro-look, pose changes, and speech animation keep Sky from appearing frozen.
- Browser speech synthesis and recognition provide a low-friction conversational layer.
- Pause and end-game states release camera and microphone resources.
- Sky's announced move remains visible until the physical board is synchronized.

2. Prototype architecture

The local prototype uses Flask for the application boundary, python-chess for legal state transitions, Stockfish for move choice, and browser APIs for camera, speech, and animation. Camera processing runs client-side so video does not leave the laptop.

Layer	Current implementation	Role
Experience	HTML/CSS/JavaScript	Sky, calibration, turn controls, voice
Vision	Browser Canvas	Perspective sampling and square-change features
Game state	python-chess	Legality, position, notation, outcomes
Decision	Stockfish 18	Bounded-strength strategic move selection
Transport	Flask JSON API	Start, move, legal moves, talk, reset

Vision pipeline tested

Four manually selected corners define a perspective mapping. Each square is sampled for color, edge strength, and luminance texture. A temporal baseline estimates normal camera noise. Whole-board median shifts compensate for exposure changes. Settled changes are compared with legal origin-destination pairs, and the strongest legal hypothesis is submitted only when confidence clears a margin.

Privacy property	The live camera stream is processed in the browser. The Flask application receives chess coordinates, not video frames.

3. What failed - and why

Repeated physical tests exposed a gap between a plausible demo algorithm and a dependable sensing system. The detector sometimes reported changes while the board was untouched; at other times it missed a real pawn move. Threshold changes reduced one error class while increasing another.

Observed failure modes

- **Auto-exposure drift:** a global camera adjustment changes many square statistics at once.
- **Shadows and hand occlusion:** the act of moving a piece temporarily affects squares unrelated to the final move.
- **Perspective sampling error:** a small corner-click error shifts feature windows away from a piece’s visual center.
- **Low visual separability:** some occupied and empty squares have similar average appearance, especially with patterned boards.
- **Single-view ambiguity:** a tall piece can overlap neighboring regions while its base remains difficult to isolate.
- **Device variability:** browser camera processing, resolution, focus, and lighting differ across machines.

Interpretation

This is not fundamentally a chess-engine problem. It is a perception and calibration problem. Legal-move constraints are useful after the vision system produces trustworthy occupancy evidence, but legality cannot repair a weak observation model. The prototype therefore should be presented as an interaction and systems-research artifact, not as completed automatic board recognition.

4. Proposed robust architecture

Stage	Method	Acceptance test
Geometry	ArUco markers or automatic grid-line detection; saved homography	All 81 intersections remain within 3 px across a session
Occupancy	Per-square empty/occupied classifier trained on captured board crops	>99.5% square-state accuracy on held-out sessions
Color	Piece-color classifier on occupied squares, using lower piece/base regions	>99% white/black accuracy under target lighting
Temporal state	Hand-present model plus stable-before/stable-after capture	No move inference while a hand intersects the board
Move inference	Difference between classified board states constrained by python-chess	>99% legal move accuracy across scripted games
Recovery	Spoken or typed correction without corrupting internal state	User can recover in one interaction

Dataset before algorithm

The next milestone is a diagnostic capture mode, not another threshold. It should save rectified before, hand-present, and after images; 64 labeled square crops; exposure metadata; predicted occupancy; and the user-confirmed move. Data should cover multiple daylight conditions, board positions, piece sets, and camera placements. Evaluation must be session-separated so the model cannot memorize one setup.

Proposed success criteria

- Zero false moves during a continuous 30-minute stationary-board test.
- At least 99% correct recognition over 200 legal moves across five sessions.
- Median confirmation latency below 2 seconds after the hand leaves the board.
- No silent state divergence; uncertain observations always request confirmation.

5. Research notebook

Iteration	Hypothesis	Result / learning
A	Four-corner perspective correction makes square changes measurable.	Useful foundation, but manual click error remains important.
B	Average RGB and edge differences identify lifted and placed pieces.	Works in controlled moments; unstable under camera adaptation.
C	Per-square noise thresholds reduce false positives.	Improves calibration, but does not model occupied vs. empty states.
D	Whole-board median compensation removes exposure drift.	Suppresses common-mode changes; local shadows still dominate.
E	Legal-move scoring disambiguates noisy change candidates.	Helpful only when both origin and destination evidence are reliable.
Next	Learn occupancy and hand state from captured examples.	Required before claiming automatic physical-board play.

Ethical presentation

The separate scripted demo communicates the intended interaction without implying that the live camera detector has achieved production reliability. Screen recordings should use the words “concept demo” or “scripted prototype.” This preserves the value of the design exploration while making the research status legible.

6. Conclusion

Sky Chess Lab demonstrates a compelling interaction model: a conversational AI can inhabit the screen while sharing attention with a physical object in the room. The prototype also establishes a clear technical boundary. Embodied intelligence depends on dependable perception, and dependable perception requires data, evaluation, and uncertainty handling - not only increasingly elaborate heuristics. The next phase is a small, instrumented computer-vision study built around occupancy classification and stable state transitions.

Artifact record

Author: Fatima Aguilar
Studio: Decision Systems Lab
Repository: https://github.com/fa366193/sky-chess-lab
Prototype stack: Flask, JavaScript, Browser Media APIs, python-chess, Stockfish
Document date: August 2026